

gather/linear

An AgentMPI collective scaling analysis

Abstract

The linear gather is the flat fan-in: every non-root rank sends its contribution directly to the root, and the root performs $p - 1$ wildcard receives and files each arrival by its source. It costs $p - 1$ messages and $(p - 1)n$ tokens, and the sweep confirms both at all twenty-one measured sizes — 7 messages at $p = 8$, 31 at $p = 32$ — which is the minimum any gather can achieve, since every rank’s contribution must cross the network once. What the family view shows is where that minimum is paid: the root, alone, and serially. At $p = 32$ the root’s 31 receives account for 0.255 s of the 0.26 rank-seconds of receive-blocking in the whole collective, 98 % of it in one rank out of thirty-two, and the viewer screenshot is a single lane of thick bars beside thirty-one lanes carrying one tick each. That is why the closed form prices the critical path at $p - 1$ rather than 1, and it is why the trace’s own `rounds` field — which reports 1 at every size — understates the cost. The more important observation for agent ranks is what *did not* happen: all 31 receives are logged `admitted: false` with `admitted_tokens: 0` and the root’s context high-water is 0, because a gather returns handles and lets the root decide what to materialise. A gather that admitted its pn tokens would make the root’s context window the scalability limit of the whole harness, and in `real-sw-p8-full` that would have been 29,847 tokens of a 120,000-token budget for a single fan-in at $p = 8$.

1 What this family is

The `gather` collective under the `linear` algorithm, measured across 21 runs at 13 distinct process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [\[ok\]](#) All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [\[note\]](#) Wall times span 0.011–0.083 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

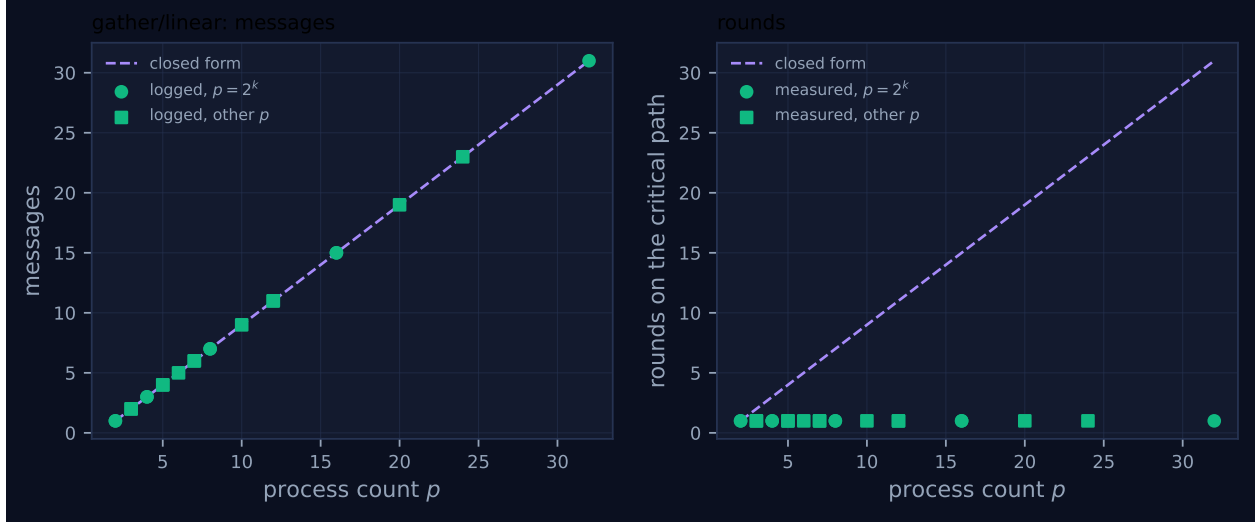


Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective’s own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	mb-free	1	1	1	1	1	0.012	✓	✓
2	mb-smoke	1	1	1	1	1	0.012	✓	✓
3	mb-free	2	2	2	1	2	0.013	✓	✓
3	mb-smoke	2	2	2	1	2	0.011	✓	✓
4	mb-free	3	3	3	1	3	0.014	✓	✓
4	mb-smoke	3	3	3	1	3	0.017	✓	✓
5	mb-free	4	4	4	1	4	0.013	✓	✓
5	mb-smoke	4	4	4	1	4	0.027	✓	✓
6	mb-free	5	5	5	1	5	0.025	✓	✓
7	mb-free	6	6	6	1	6	0.032	✓	✓
7	mb-smoke	6	6	6	1	6	0.016	✓	✓
8	mb-free	7	7	7	1	7	0.027	✓	✓
8	mb-smoke	7	7	7	1	7	0.022	✓	✓
10	mb-free	9	9	9	1	9	0.028	✓	✓
12	mb-free	11	11	11	1	11	0.033	✓	✓
12	mb-smoke	11	11	11	1	11	0.050	✓	✓
16	mb-free	15	15	15	1	15	0.050	✓	✓
16	mb-smoke	15	15	15	1	15	0.044	✓	✓
20	mb-free	19	19	19	1	19	0.071	✓	✓
24	mb-free	23	23	23	1	23	0.072	✓	✓
32	mb-free	31	31	31	1	31	0.083	✓	✓

4 How the algorithm scales

The root’s branch is `for _ in range(p - 1)` wildcard receives, each filed by `msg.source` into an output list; the non-root branch is one `_csend` to the root. So the message count is $p - 1$ and the volume $(p - 1)n$, which is `FORMULAS["gather", "linear"]`, and the fourth term — fold depth —

is zero, because a gather composes nothing. The closed form gives $p - 1$ rounds on the critical path, and that is the number worth understanding, because it is the whole difference between this algorithm and its binomial sibling: the two move identical message counts, and only the shape of the fan-in differs.

The message panel of Figure 1 is the line $p - 1$ and matches at every one of the twenty-one rows, from 1 at $p = 2$ to 31 at $p = 32$; the self-reported token volume is also $p - 1$ at every size, the benchmark's `f"r{rank}"` contribution tokenising to one token. There is no shape to explain. The round panel, however, shows a measured 1 against a predicted $p - 1$ at every size from $p = 3$ upward, and that gap is not a defect in the algorithm but a disagreement about what “round” means, which the family view is the first thing to surface.

The implementation writes `tr.stats.rounds = 1`: one logical phase, in which all $p - 1$ sends happen and all $p - 1$ receives happen. The closed form writes $p - 1$: the root cannot overlap its receives, so $p - 1$ message costs lie in sequence on the path from the first send to the last placement. The module is not consistent about which convention it records — `bcast/flat`, the structurally identical fan-out, records `1 if r != root else p - 1`, taking the serialised reading and making it rank-dependent, which is the honest choice and the one that would have made this column agree. So the `rounds` column here should be read as “phases the algorithm believes it executed” rather than “message costs on the critical path”.

The log settles which quantity is the real one. Summing the `latency_s` field of every `msg.recv` record at $p = 32$ gives 0.26 rank-seconds of receive-blocking, of which 0.255 belongs to rank 0's thirty-one receives; the aggregate grows as $p^{1.28}$ ($R^2 = 0.978$), essentially linearly, because there is only ever one rank waiting. The viewer screenshot for `mb-free__coll-gather-linear-32` draws exactly that: rank 0's lane carries a run of thick bars spread across the full 0.09s span — bars, in the viewer's encoding, being glyphs with extent — while all thirty-one other lanes carry a single thin tick, their one send. Nothing else in the picture has duration. A reader who took the `rounds` column at face value would conclude this collective completed in one step; the screenshot shows thirty-one steps stacked in one lane.

5 Powers of two and everything else

There is nothing power-of-two about this algorithm. The root's loop is `range(p - 1)` and the receive is a wildcard, so no rank's identity enters any arithmetic and the code path at $p = 7$ is byte-for-byte the path at $p = 8$. The family view confirms it: the squares in Figure 1 lie on $p - 1$ exactly as the circles do — 2, 4, 5, 6, 9, 11, 19 and 23 at $p = 3, 5, 6, 7, 10, 12, 20, 24$ — there are no red crosses, so the collective's self-reported count equalled the fabric's logged traffic at all twenty-one sizes, and the eight sizes measured in both `mb-free` and `mb-smoke` agree on every integer, differing only in wall time (0.027s against 0.022s at $p = 8$).

That makes this family the control for its op. When `gather/binomial` shows something odd at a non-power-of-two size — and it does, in its volume term — the question is whether the oddity belongs to the tree arithmetic or to the measurement apparatus. A sibling with no bit arithmetic and no disagreement at any size answers that: the apparatus is fine.

One asymmetry between the two algorithms is worth recording here even though the sweep cannot show it, because it is a semantic difference rather than a cost difference and it hides at exactly this boundary. `gather()` chooses its algorithm as `"linear" if p <= 4 else "binomial"` when the caller does not name one. The `linear` branch honours the caller's `admit` argument, passing

it through to `comm.recv(..., admit=admit)`; the `binomial` branch hard-codes `admit=False` on its child receives and ignores the argument entirely. So `comm.gather(x, admit=True)` charges the root’s context account at $p = 4$ and silently does not at $p = 5$. The sweep cannot expose this because it names the algorithm explicitly and leaves `admit` at its default, so every receive in all twenty-one runs is logged `admitted: false`; the finding comes from reading the two branches side by side, which is what a family document is for.

6 When to choose this algorithm

The choice between `linear` and `binomial` is depth against fan-out at equal message count, and it is the cleanest crossover in the sweep because the message counts are identical — $p - 1$ in both, at every one of the twenty-one sizes in both families. Nothing is traded away in traffic; what changes is where the traffic converges and how much volume passes through intermediate ranks.

`linear` wins on volume, and by exactly the ratio the closed forms give: $(p - 1)n$ against $np[\log_2 p]/2$, which at $p = 32$ is 31 units against 80 — both figures confirmed by the collectives’ self-reports in the two sweeps. It loses on depth by the same construction: $p - 1$ against $\lceil \log_2 p \rceil$, or 31 against 5 at $p = 32$. Running that through `cost.predict` at $n = 4,096$ tokens per contribution gives 15.97 s of critical path for `linear` against 2.58 s for `binomial`, and 126,976 against 327,680 tokens of volume: 6.2 times slower for 2.6 times fewer tokens. `cost.best_algorithm` accordingly answers `binomial` on the time objective and `linear` on the volume objective at every (p, n) pair I evaluated, which is an unusually honest crossover — the two objectives genuinely disagree and the model says so rather than picking.

For agent ranks the decision usually goes to `binomial`, and the reason is not the token count but the root. Under `linear` the root performs $p - 1$ receives itself; under `binomial` it performs $\lceil \log_2 p \rceil$, and the sweep measures the difference at $p = 32$ as 0.255 s of root blocking against 0.049 s, a factor of 5.2. If the root is an agent rather than a thread, each of those receives is a turn in which it is doing nothing else, and $p - 1$ of them is a serial section in Amdahl’s sense — the phase that “must be done by one rank” and does not shrink with p . `linear` is the right choice when the volume penalty is what binds: small p , large contributions, and a harness that is going to read everything anyway.

The larger argument, and the one this family is best placed to make, is about *how* the contributions arrive rather than which algorithm delivers them. A gather of p artefacts of n tokens presents the root with pn tokens, and for realistic p and n that exceeds any context window — which would make the root’s context the scalability limit of the whole harness, independent of algorithm. AgentMPI’s answer is to return handles: `mode=AUTO` sends large contributions by rendezvous and `admit=False` keeps them out of the root’s context, so the root receives a list of references and materialises what it needs. The sweep shows the admission half of that mechanism working — all 31 receives at $p = 32$ logged `admitted: false`, `admitted_tokens: 0`, and every rank’s finalisation reporting a context high-water of 0 — but cannot show the transport half, because the benchmark launched with `eager_limit=10**9` and `Mode.AUTO` therefore chose eager for every message. The transport half is visible in `real-sw-p8-full1`: all fourteen `gather` sends there are `mode: rendezvous`, the largest single contribution is 13,206 tokens, the `collect-r1` gather moved 29,847 tokens in seven messages, and the root’s context high-water is 0 against a 120,000-token budget. Admitting that one fan-in by value would have consumed a quarter of the root’s window; two rounds of it would have consumed half.

7 Threats to this reading

No agents took part, so nothing here is about model behaviour. No executor is recorded, the viewer reports zero agent calls, and the wall times of 0.011–0.083s are SQLite writes and event-log appends. The root-serialisation measurements — 0.255s across 31 receives, the $p^{1.28}$ fit — are fabric behaviour under thread contention. They demonstrate the *shape* of the fan-in, which is what the algorithm determines, and they say nothing about what a fan-in costs when each contribution is a model’s output.

The contributions are one token each, which suppresses the effect the collective exists to manage. The benchmark gathers `f"r{rank}"`, so the total volume at $p = 32$ is 31 tokens and no context budget could be threatened by it. Every statement above about context pressure is therefore an argument from the closed form and from the real run, not a measurement from this family. A sweep with realistic payloads would be a different and more informative experiment; this one measures the pattern.

The transport mode is an artefact of the launch configuration. All messages are eager because `eager_limit=10**9` forces `Mode.AUTO` to eager for any payload, and `strict_context=False` disables the admission checks. The `admitted: false` records are still meaningful — they show the collective passing `admit=False` through to the receive, which is a code-path property — but a reader should not infer that `Mode.AUTO` would have chosen rendezvous here, because it was never given the chance.

The round-count gap is a definitional disagreement, not a measurement error, and I have not proven which convention is intended. The implementation says 1, the closed form says $p - 1$, and `bcast/flat` in the same module says $p - 1$ at the root for the mirror-image operation. My reading is that the serialised convention is the useful one and that this branch should follow `bcast/flat`; someone who intended rounds to mean “phases” would say the closed form is what needs changing. Either way the two do not currently agree, and no tooling in the repository compares them — `bench_collectives` records both and tests only messages and fold depth.

The admit asymmetry is read from source, not observed. No run in this sweep passes `admit=True`, so the claim that `binomial` ignores it is a code reading. It is a short reading — one branch passes the argument, the other passes a literal `False` — but it is not a measurement, and a test would settle it.